

Classical Machine Learning

The Normal Equation, sigmoid & log loss, Gini impurity, distance metrics, margins, and PCA

Concept Guide 3 of 9 | Read before: *Classical ML Zero-to-Hero lab* | Companion to the *Zero-to-Hero* series

1. Linear Regression: The Normal Equation

Linear regression fits a straight line (or hyperplane) that minimizes squared error. Unlike neural networks, it has an exact, one-shot algebraic solution — no iterative training needed.

FORMULA — The Normal Equation

$$w = (X^T X)^{-1} X^T y$$

X is the design matrix (rows=examples, cols=features, plus a column of 1s for the bias)

MENTAL MODEL

This formula finds the exact w that minimizes total squared error in one step, using only linear algebra. Gradient descent (used for regression too, and for everything nonlinear) finds the SAME answer iteratively — useful when the exact matrix inversion becomes too expensive at huge scale.

COMMON MISUNDERSTANDINGS

- The Normal Equation requires $(X^T X)$ to be invertible — this fails when features are perfectly correlated (redundant) or when there are more features than examples.
- It doesn't scale to millions of features due to the cost of matrix inversion, which is why gradient descent is the default at large scale.

PRACTICE THIS → [Classical_ML_Zero_to_Hero.ipynb Ch.2](#); [NumPy_Zero_to_Master.ipynb Ch.11](#)

2. Logistic Regression: Sigmoid and Log Loss

For classification, we need a bounded probability, not an unbounded number. Logistic regression squashes a linear combination through the sigmoid function, then trains with log loss (cross-entropy) instead of squared error.

FORMULA — Sigmoid and log loss

$$\text{sigma}(z) = 1 / (1 + e^{-z})$$
$$\text{log_loss} = -[y \cdot \log(p) + (1-y) \cdot \log(1-p)]$$

$p = \text{sigma}(z)$ is the predicted probability; y is the true label (0 or 1)

WORKED EXAMPLE — Why log loss punishes confident wrong answers hard

True label $y=1$. Model predicts $p=0.9$ (confident, correct): $\text{loss} = -\log(0.9) = 0.105$ (small)

True label $y=1$. Model predicts $p=0.1$ (confident, WRONG): $\text{loss} = -\log(0.1) = 2.303$ (large)

The loss grows without bound as a confident wrong prediction approaches 0 probability.

MENTAL MODEL

Log loss is designed so that being confidently wrong is punished far more than being uncertain. This pushes the model toward well-calibrated probabilities, not just correct labels.

PRACTICE THIS → [Classical_ML_Zero_to_Hero.ipynb Ch.3](#)

3. Decision Trees: Gini Impurity and Entropy

A decision tree picks, at each split, the question that most reduces 'impurity' — how mixed the classes are in a node. Two common impurity measures:

FORMULA — Gini impurity and entropy

$$\text{Gini}(\text{node}) = 1 - \sum(p_i^2)$$
$$\text{Entropy}(\text{node}) = -\sum(p_i \cdot \log_2(p_i))$$

p_i is the fraction of class i in that node; both are 0 for a pure node

WORKED EXAMPLE — Worked example: a node with 8 items, 6 class A and 2 class B

$$p_A = 6/8 = 0.75, p_B = 2/8 = 0.25$$

$$\text{Gini} = 1 - (0.75^2 + 0.25^2) = 1 - (0.5625 + 0.0625) = 1 - 0.625 = 0.375$$

A perfectly pure node (all one class) has $\text{Gini} = 0$.

A perfectly mixed 50/50 node has the maximum $\text{Gini} = 0.5$ (for 2 classes).

MENTAL MODEL

Think of Gini impurity as 'if I randomly picked two items from this node, how often would they be different classes?' Lower means purer, and purer nodes make better, more confident predictions.

PRACTICE THIS → [Classical_ML_Zero_to_Hero.ipynb Ch.4-5](#)

4. k-Nearest Neighbors and k-Means: Distance-Based Learning

Both algorithms rely entirely on a distance metric — usually Euclidean distance — to decide what's 'close.'

FORMULA — Euclidean distance

$$d(a, b) = \sqrt{\sum (a_i - b_i)^2}$$

the straight-line distance between two points in n-dimensional space

DIAGRAM — k-Means: the iterative loop

1. Assign each point to its nearest centroid
2. Move each centroid to the MEAN of its assigned points
3. Repeat steps 1-2 until centroids stop moving

COMMON MISUNDERSTANDINGS

- Both k-NN and k-Means are highly sensitive to feature SCALE — a feature ranging 0-1000 dominates distance even if a 0-1 feature is more predictive. Always standardize first (see the Math & Statistics guide).
- k-Means requires choosing k in advance; it cannot discover the 'right' number of clusters on its own.

PRACTICE THIS → [Classical_ML_Zero_to_Hero.ipynb Ch.6-7](#)

5. Support Vector Machines: Margin Maximization

An SVM finds the decision boundary with the widest possible gap (margin) between the two classes. The points that define this margin are called 'support vectors.'

DIAGRAM — Maximum margin

class A	margin	class B
* *	<----->	o o
*	boundary	o
*	^	o

the widest gap between the classes

MENTAL MODEL

A wider margin tends to generalize better to new data — it's not just separating the training points, it's separating them with maximum 'breathing room.' The kernel trick (used for non-linearly-separable data) implicitly projects data into a higher dimension where a straight-line boundary becomes possible, without ever computing that higher-dimensional space explicitly.

PRACTICE THIS → [Classical_ML_Zero_to_Hero.ipynb Ch.8](#)

6. PCA: Variance and Eigenvectors

Principal Component Analysis finds new axes that capture the most variance in the data, letting you compress many correlated features into a few.

FORMULA — The core idea

Compute the covariance matrix of the data.
Its eigenvectors are the principal components (new axes).
Its eigenvalues tell you how much variance each axis captures.

see the NumPy & Linear Algebra guide for the eigenvector equation itself

WORKED EXAMPLE — Reading 'explained variance ratio'

If PC1 has `explained_variance_ratio = 0.85` and PC2 has 0.12,
then just 2 components capture $0.85 + 0.12 = 97\%$ of the original information
— even if the original data had, say, 20 features.

PRACTICE THIS → [Classical_ML_Zero_to_Hero.ipynb Ch.9](#)